人工智能站点聚类实验报告

运行环境与操作指令

运行环境

- 硬件信息
  - 显卡：RTX 3090
  - 显存：24GB
  - CPU型号：AMD EPYC 7642 48-Core Processor
  - CPU核心数：24核
  - 内存：80GB
- 操作系统：ubuntu20.04
- 运行镜像：Miniconda/conda3.8/cuda11.3
- 编辑器：JupyterLab
- 编程语言：Python 3.8.10
- 依赖包：requirement.txt

操作指令

依赖包安装

1. 实验所需硬件平台使用的是AutoDL的硬件设备，其中镜像信息使用的是AutoDL的基础镜像

   Miniconda / conda3 / 3.8(ubuntu20.04) / 11.3

2. 创建完主机实例后进入JupyterLab，安装依赖包
   因为之前已经进行过依赖包的安装，我们将所有依赖包的信息进行了导出

   ```
   pip freeze > requirement.txt
   ```

```
root@autodl-container-ac78118c3c-3cdd422e:~# pip freeze
absl-py==1.2.0
anyio @ file:///home/conda/feedstock_root/build_artifacts/anyio_1666191106763/work/dist
appdirs==1.4.4
argon2-cffi @ file:///home/conda/feedstock_root/build_artifacts/argon2-cffi_1640817743617/work
argon2-cffi-bindings @ file:///tmp/build/80754af9/argon2-cffi-bindings_1644569684262/work
astroid==2.12.13
asttokens @ file:///home/conda/feedstock_root/build_artifacts/asttokens_1669740551754/work
attrs @ file:///home/conda/feedstock_root/build_artifacts/attrs_1659291887007/work
autopep8==1.6.0
Babel @ file:///home/conda/feedstock_root/build_artifacts/babel_1667688356751/work
backcall @ file:///home/conda/feedstock_root/build_artifacts/backcall_1592338393461/work
backports.functools-lru-cache @ file:///home/conda/feedstock_root/build_artifacts/backports.functools_lru_cache_1618230623929/work
basemap==1.3.6
basemap-data==1.3.2
beautifulsoup4 @ file:///home/conda/feedstock_root/build_artifacts/beautifulsoup4_1649463573192/work
bleach @ file:///home/conda/feedstock_root/build_artifacts/bleach_1656355450470/work
brotlipy==0.7.0
cachetools==5.2.0
certifi==2022.9.24
cffi @ file:///tmp/build/80754af9/cffi_1625807838443/work
chardet @ file:///tmp/build/80754af9/chardet_1607706746162/work
click==8.1.3
cloudpickle==2.2.0
conda==22.9.0
conda-package-handling @ file:///tmp/build/80754af9/conda-package-handling_1618262148928/work
cryptography @ file:///tmp/build/80754af9/cryptography_1616769286105/work
cubinlinker-cu11==0.3.0
cuda-python==11.7.0
cudf-cu11==22.10.0
cuml-cu11==22.10.0
cupy-cuda112==10.6.0
cycler==0.11.0
Cython==0.29.32
dask==2022.9.2
dask-cuda==22.10.0
dask-cudf-cu11==22.10.0
debugpy==1.6.2
decorator @ file:///home/conda/feedstock_root/build_artifacts/decorator_1641555617451/work
defusedxml @ file:///home/conda/feedstock_root/build_artifacts/defusedxml_1615232257335/work
dill==0.3.6
distributed==2022.9.2
docstring-to-markdown==0.11
entrypoints @ file:///home/conda/feedstock_root/build_artifacts/entrypoints_1643888246732/work
executing @ file:///home/conda/feedstock_root/build_artifacts/executing_1667317341051/work
```

之后在安装python依赖包时，可以直接使用以下指令直接进行安装

```
pip install -r requirement.txt
```

JupyterLab 插件安装

为了能够显示代码的运行时间，需要安装一个JupyterLab插件jupyter-excute-time

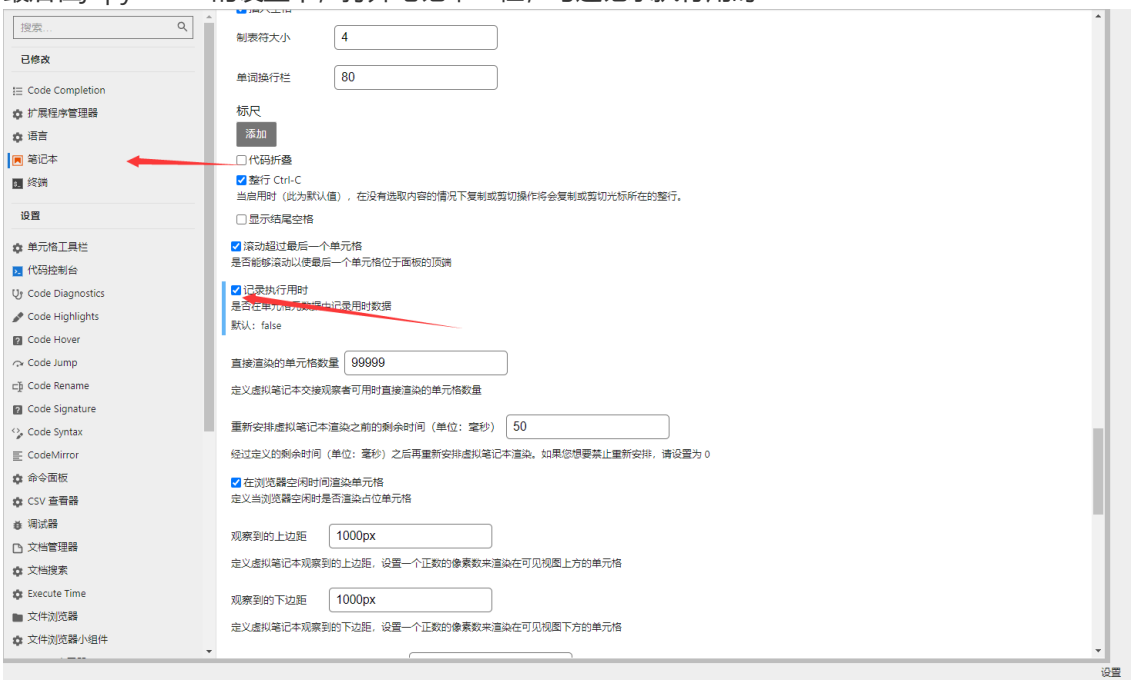
1. 首先我们需要安装依赖项，npm和nodejs两个库

```
pip install npm
conda install -c conda-forge nodejs
```

2. 之后安装jupyter-excute-time插件

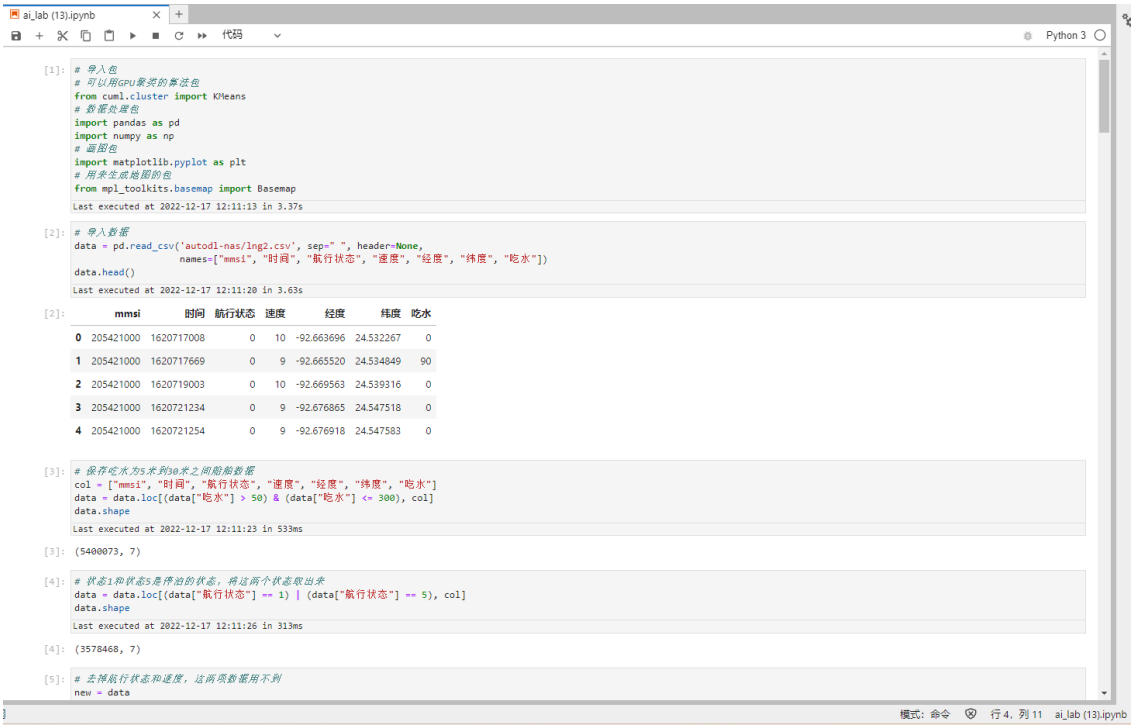
```
pip install jupyterlab_execute_time
```

3. 最后在JupyterLab的设置中，打开笔记本一栏，勾选记录执行用时



程序运行

1. 将代码文件用JupyterLab打开
   代码文件为压缩包中ai_lab.ipynb文件
   其中，导入lng.csv数据集的相对目录位置为`autodl-nas/lng2.csv`



2. 依次执行单元格中的代码，得到的输出文件中
   `ai_lab_output_data`为中间过程分析文件

   - in_rate:船只进口比率
   - out_rate:船只出口比率
   - no_rate:船只停靠比率

   `final_data`为结果文件

代码运行截图

主要运行截图

- 数据预处理截图

[3]:
```
# 保存吃水为5米到30米之间船舶数据
col = ["mmsi", "时间", "航行状态", "速度", "经度", "纬度", "吃水"]
data = data.loc[(data["吃水"] > 50) & (data["吃水"] <= 300), col]
data.shape
```
Last executed at 2022-12-17 12:11:23 in 533ms

[3]: (5400073, 7)

[4]:
```
# 状态1和状态5是停泊的状态，将这两个状态取出来
data = data.loc[(data["航行状态"] == 1) | (data["航行状态"] == 5), col]
data.shape
```
Last executed at 2022-12-17 12:11:26 in 313ms

[4]: (3578468, 7)

[5]:
```
# 去掉航行状态和速度，这两项数据用不到
new = data
new = new.drop("航行状态", axis=1)
new = new.drop("速度", axis=1)
new.head()
```
Last executed at 2022-12-17 12:11:28 in 164ms

[5]:

	mmsi	时间	经度	纬度	吃水
71	205421000	1621112664	-93.690987	29.430099	90
78	205421000	1621115364	-93.691147	29.429800	90
80	205421000	1621115724	-93.691147	29.429951	90
81	205421000	1621115903	-93.691132	29.430000	90
84	205421000	1621116270	-93.691170	29.429983	90

[6]:
```
# 重新对new和data索引
new.index = range(1, len(new) + 1)
data.index = range(1, len(data) + 1)
```
Last executed at 2022-12-17 12:11:30 in 4ms

- 聚类算法运行截图

```
opt_list = []  # 保存簇从150到200，每个簇都会跑3遍，保存最优的score
lable_list = []  # 保存最优的分类标签
center_list = []  # 保存簇心坐标数据
iter_num = 500  # 设置最大迭代次数
for clus_k in range(150, 211):
    mx = 0  # 从跑的3遍中选最大的轮廓系数
    y_ = []
    center = []
    for i in range(1, 4):
        # 为了选到最优的簇数，每个簇都跑3遍，为了减小kmeans算法中初始质心随机带来的影响
        kmean = KMeans(n_clusters=clus_k, max_iter=iter_num)
        kmean.fit_predict(train)
        s = -kmean.score(train)
        if i == 1:
            mx = s
            y_ = kmean.labels_.tolist()
            center = kmean.cluster_centers_.tolist()
        if mx > s:  # score越小越好
            mx = s
            y_ = kmean.labels_.tolist()
            center = kmean.cluster_centers_.tolist()
    opt_list.append(mx)
    lable_list.append(y_)
    center_list.append(center)
```
Last executed at 2022-12-17 12:22:00 in 8m 31.34s

代码运行时长：8分31.34s

- 站点分析截图

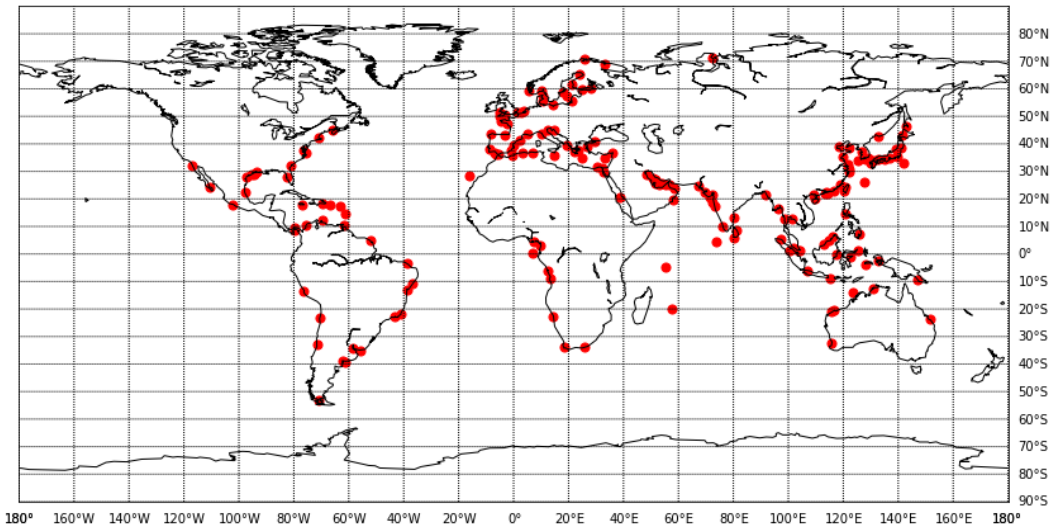
```python
in_rate = []
out_rate = []
no_rate = []
for i in range(0, 183):
    # 第一个索引是分组编号，第二个索引为0为分组标号，为1是分组的数据
    g = list(newgroup)[i][1]
    # 船最早进来的数据
    firstin = g.groupby("mmsi", as_index=False).agg({'时间': 'min'})
    # 加上其入站的吃水深度
    firstin = pd.merge(firstin, g)
    # 删除不用的项目，只保留mmsi、入站吃水
    firstin = firstin.drop(columns=["时间", "经度", "纬度", "站点编号"])
    # 改名
    firstin.columns = ["mmsi", "入站吃水"]
    # 船最晚出去的数据
    lastout = g.groupby("mmsi", as_index=False).agg({'时间': 'max'})
    # 加上其入站的吃水深度
    lastout = pd.merge(lastout, g)
    # 删除不用的项目，只保留mmsi、入站吃水
    lastout = lastout.drop(columns=["时间", "经度", "纬度", "站点编号"])
    # 改名
    lastout.columns = ["mmsi", "出站吃水"]
    # 将入站吃水和出站吃水合并
    res = pd.merge(firstin, lastout)
    res["变化(m)"] = (res["出站吃水"] - res["入站吃水"]) / 10
    temp = []
    for i in range(0, res.shape[0]):
        if res['变化(m)'][i] < 0:
            temp.append(-1)  # 卸货
        elif res['变化(m)'][i] == 0:
            temp.append(0)  # 停靠
        else:
            temp.append(1)  # 装货
    res["变化行为"] = temp
    # 计算船只变化行为的比率
    boat_rate = res["变化行为"].value_counts(normalize=True)  # boat_rate返回回来为Series类型
    if -1 in boat_rate:
        out_rate.append(boat_rate[-1])
    else:
        out_rate.append(0)
    if 0 in boat_rate:
        no_rate.append(boat_rate[0])
    else:
        no_rate.append(0)
    if 1 in boat_rate:
        in_rate.append(boat_rate[1])
    else:
        in_rate.append(0)
```

Last executed at 2022-12-17 12:54:13 in 42.62s

- 地图标点截图

```python
# 在世界地图上标出站点
plt.figure(figsize=(14, 8))
m = Basemap()   # 创建一个地图
m.drawcoastlines()   # 画海岸线
# m.drawcounties(linewidth=1)     # 画国家分界线
parallels = np.arange(-90., 90., 10.)  # 这两行画纬度，范围为[-90,90]间隔为10
m.drawparallels(parallels, labels=[False, True, True, False])
meridians = np.arange(-180., 180., 20.)  # 这两行画经度，范围为[-180,180]间隔为10
m.drawmeridians(meridians, labels=[True, False, False, True])
lon, lat = m(lon, lat)  # lon, lat为给定的经纬度，可以使单个的，也可以是列表
m.scatter(lon, lat, s=50, c="red")  # 标注出所在的点，s为点的大小，还可以选择点的性状和颜色等属性
plt.show()   # 显示图像
```

Last executed at 2022-12-17 12:53:20 in 379ms



代码性能分析

- 最初我们选择使用的是sklearn的机器学习库，用于将数据进行聚类，但通过很多次试验和了解后我们发现，sklearn库对于大量数据的处理效果并不理想。这是由于sklearn无法支持GPU运算，仅支持CPU运算而导致的。而我们已有的硬件设备中，并没有理想的多核CPU集群运算条件，而sklearn又无法运用到比较优势的GPU硬件条件。通过测试，在计算一次聚类结果，通常需要花费20分钟左右。

  尝试解决：英特尔平台对于Scikit-learn 应用程序，有一个加速器库`sklearnex`，可以带来10-100倍的加速，我们在英特尔平台尝试了运用该库进行加速运算，但效果仍然不理想。

- 随后在通过查阅资料和了解后，我们找到Nvidia有一个专门用于机器学习的库，支持使用GPU进行运算
  `cuml`:接口与sklearn库提供的接口使用方法基本一直一致
  [cuml官方文档]
  通过测试，在计算一次聚类结果，cuml使用GPU只需要花费5秒左右。

预处理和算法分析

整个流程可以主要分为三大部分，`数据预处理`，`站点聚类`，`站点类型分析`。

数据预处理

总共有`7896919`条数据

1. 吃水数据处理
   通过查询资料

   > **30万吨lng船吃水(30万吨油轮吃水)**
   >
   > 来源：www.shuishangwuliu.com 时间：2022-11-01 03:15 点击：251 编辑：jing 手机版
   >
   > **1. 30万吨油轮吃水**
   >
   > 船舶吃水是指船舶在载重情况下浸入水中的深度(水线面与船底基平面之间的垂直距离).由于船舶设计中和在使用过程中通常有纵倾情况,因此每种吃水又分为艏吃水、艉吃水、平均吃水三种.艏吃水是指船舶艏部吃水深度,艉吃水是指尾部吃水深度,平均吃水就是艏艉吃水平均值。頭條萊坦
   >
   > 30万吨船吃水 空载吃水前部7米、后部9．5米,重载吃水20．85米,坦頭條萊

   我们将吃水在5m~30m的船只进行保留，将其他不符合要求的船只数据进行删除

```
# 保存吃水为5米到30米之间船舶数据
col = ["mmsi", "时间", "航行状态", "速度", "经度", "纬度", "吃水"]
data = data.loc[(data["吃水"] > 50) & (data["吃水"] <= 300), col]
data.shape
```

```
Last executed at 2022-12-17 12:11:23 in 533ms
```

```
(5400073, 7)
```

   通过处理后，剩余`5400073`条数据

2. 船只航行状态处理
   通过查询AIS航行状态

   AIS signals include a **Navigational Status** field which is reported by the vessel. This is manually set by the crew.

   The possible values and their significance are the following:

   0 = under way using engine

   1 = at anchor

   2 = not under command

   3 = restricted maneuverability

   4 = constrained by her draught

   5 = moored

   6 = aground

   7 = engaged in fishing

   8 = under way sailing

   9 = reserved for future amendment of navigational status for ships carrying DG, HS, or MP, or IMO hazard or pollutant category C, high-speed craft (HSC)

   10 = reserved for future amendment of navigational status for ships carrying dangerous goods (DG), harmful substances (HS) or marine pollutants (MP), or IMO hazard or pollutant category A, wing in ground (WIG)

   11 = power-driven vessel towing astern (regional use)

   12 = power-driven vessel pushing ahead or towing alongside (regional use)

   13 = reserved for future use

   我们认为状态1和状态5是停靠的状态，所以将不是状态1和状态5的数据进行清洗删除。

```
# 状态1和状态5是停泊的状态，将这两个状态取出来
data = data.loc[(data["航行状态"] == 1) | (data["航行状态"] == 5), col]
data.shape
```

```
Last executed at 2022-12-17 12:11:26 in 313ms
```

```
(3578468, 7)
```

   剩余 `3578468` 条数据

3. 在进行站点聚类的时候，用不到 `航行状态` 和 `速度` 这两个数据项，所以我们将这两个数据项进行了删除

```
# 去掉航行状态和速度，这两项数据用不到
new = data
new = new.drop("航行状态", axis=1)
new = new.drop("速度", axis=1)
new.head()
```

```
Last executed at 2022-12-17 12:11:28 in 164ms
```

   数据预处理完毕

站点聚类

我们使用的聚类算法是`K-Means`算法，K-Means算法需要我们在初始的时候设定聚类的簇数，而聚类的簇数也是聚类是否准确的关键，所以我们需要确定一个合适的聚类簇数，因此我们需要评价聚类的好坏。

评估方法：`Inertia指标`：每个样本与其簇质心的距离的平方和

cuml中没有Inertia属性，但是有score方法，score为Inertia值的负数

Methods

`fit` (self, X[, sample_weight])	Compute k-means clustering with X.
`fit_predict` (self, X[, sample_weight])	Compute cluster centers and predict cluster index for each sample.
`fit_transform` (self, X[, convert_dtype, ...])	Compute clustering and transform X to cluster-distance space.
`get_param_names` (self)	
`predict` (self, X[, convert_dtype, ...])	Predict the closest cluster each sample in X belongs to.
`score` (self, X[, y, sample_weight, convert_dtype])	Opposite of the value of X on the K-means objective.
`transform` (self, X[, convert_dtype])	Transform X to a cluster-distance space.

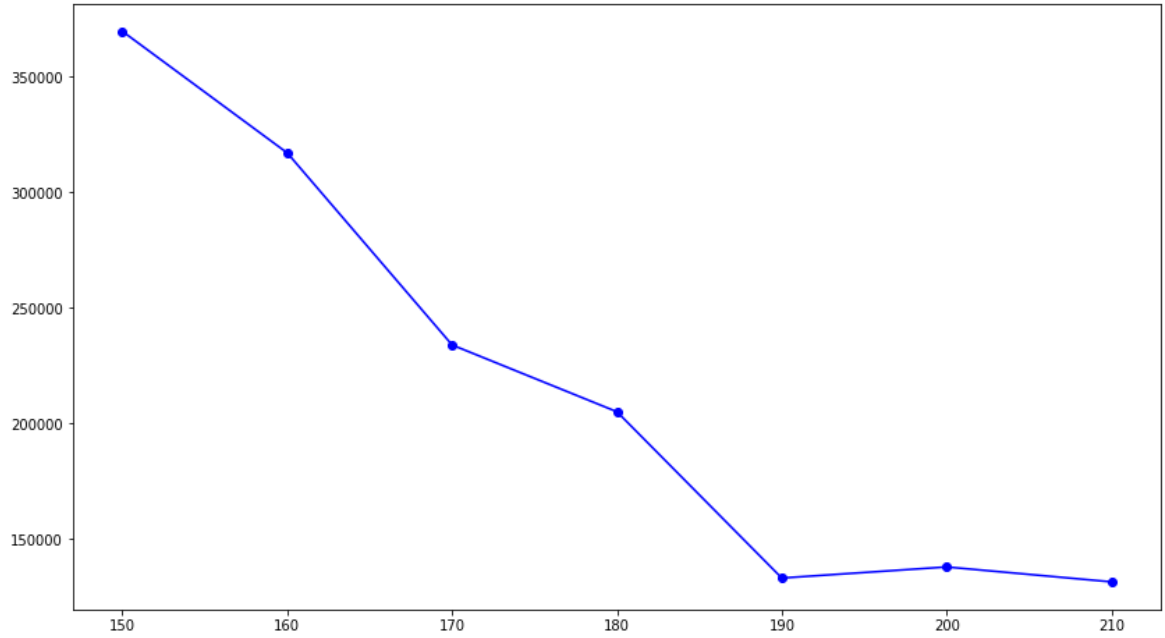
可以知道。簇数越多，指标肯定会越来越小。

我们首先设定一个大概的簇数区间，我们设定了150~210的簇数区间，每个簇数都聚类3次，目的是为了消除初始质心带来的偶然性，然后挑选聚类3次中最小的一次指标保存。

```
opt_list = []  # 保存簇从150到200，每个簇都会跑3遍，保存最优的score
lable_list = []  # 保存最优的分类标签
center_list = []  # 保存簇心坐标数据
iter_num = 500  # 设置最大迭代次数
for clus_k in range(150, 211):
    mx = 0  # 从跑的3遍中选最大的轮廓系数
    y_ = []
    center = []
    for i in range(1, 4):
        # 为了选到最优的簇数，每个簇都跑3遍，为了减小kmeans算法中初始质心随机带来的影响
        kmean = KMeans(n_clusters=clus_k, max_iter=iter_num)
        kmean.fit_predict(train)
        s = -kmean.score(train)
        if i == 1:
            mx = s
            y_ = kmean.labels_.tolist()
            center = kmean.cluster_centers_.tolist()
        if mx > s:  # score越小越好
            mx = s
            y_ = kmean.labels_.tolist()
            center = kmean.cluster_centers_.tolist()
    opt_list.append(mx)
    lable_list.append(y_)
    center_list.append(center)
Last executed at 2022-12-03 14:46:31 in 8m 17.83s
```

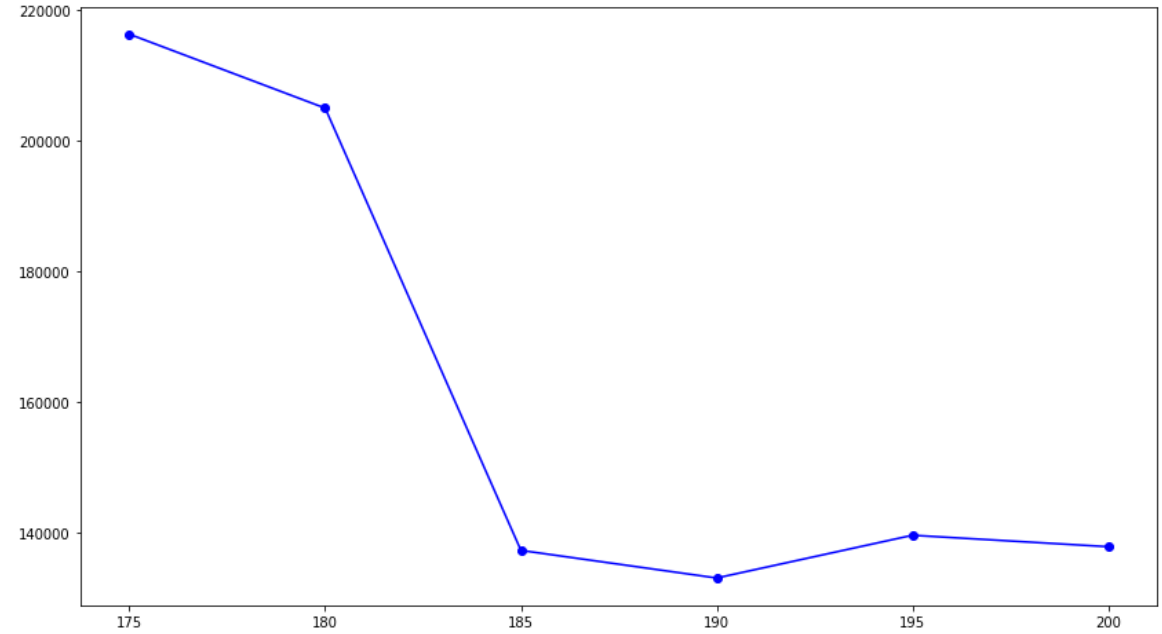
下面我们根据Inertia指标画图，寻找最佳的簇数

画出150~210簇数区间中，单位为10的图像



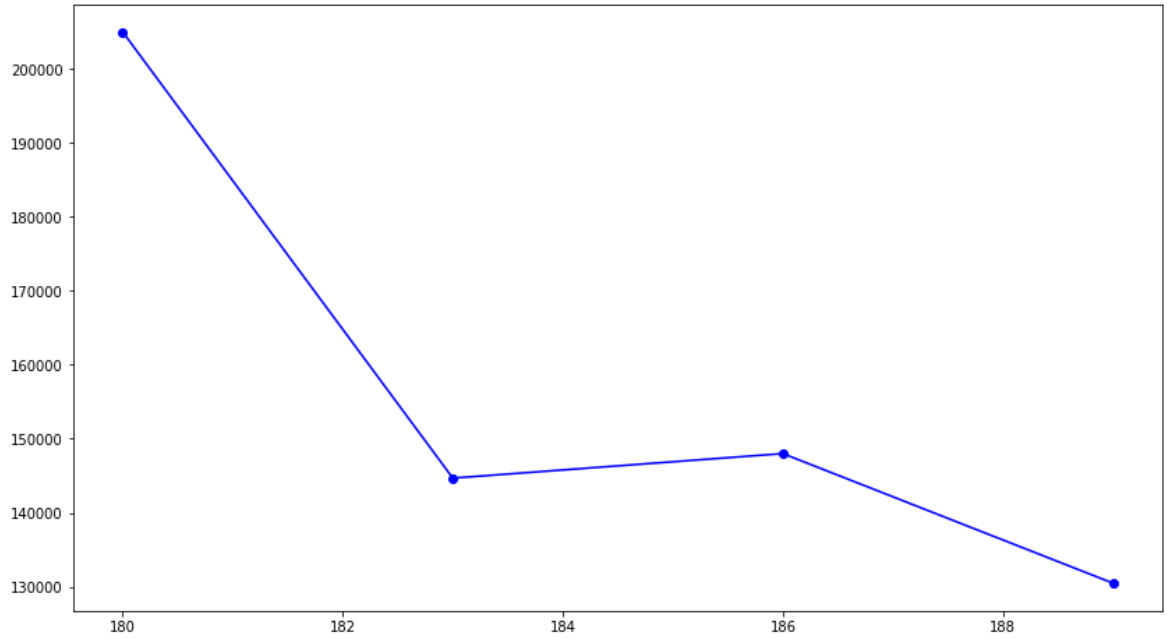
我们可以看到，190这个点为一个拐点。在190之前下降非常快，在190之后下降逐渐平缓，下降平缓则说明增加簇数带来的效果已经不明显。这说明簇数选择在190左右会比较合适，所以我们缩小范围和单位，继续寻找最佳的簇数。

画出175~200簇数区间中，单位为5的图像



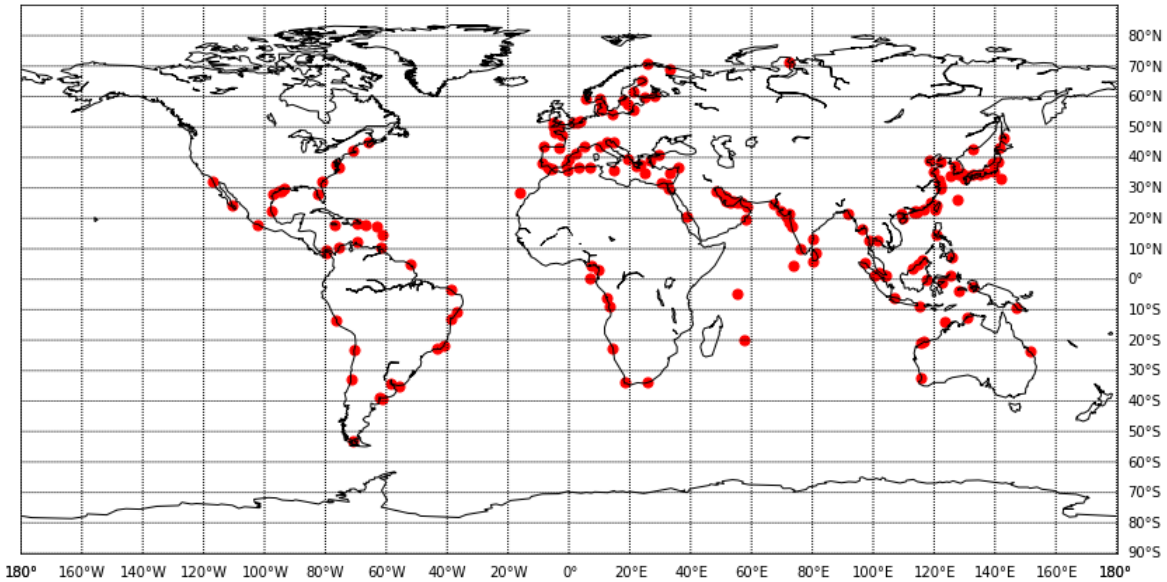
可以看到在185后面变化开始平缓，这说明簇数设置在185左右比较合适。我们进一步缩小范围。

画出180~190簇数区间中，单位为3的图像



可以看到183时为一个拐点，后面的下降变得平缓，这说明簇数设置为183会比较合适。所以我们最终确定簇数为**183**。

即确定了聚类出183个站点，我们将簇的质心作为站点的经纬度坐标。



通过将183个站点画到世界地图上，可以看到站点基本沿着海岸线，这说明聚类的效果比较好。

此时，聚类部分的工作已经完成，接下来要通过每个站点的船只数据进行对站点的分析。

站点类型分析

通过站点聚类，现在每条数据都有聚类出来的簇标签，即每条船只数据都有自己的站点编号。相反地，183个站点都有属于自己的船只数据。我们要通过这些船只数据，完成对于站点是否是LNG站点以及是否是进口站点的分析。

而这些要依靠船只数据中的吃水数据来分析出结果。

> 思路是：吃水变化有三种，减少、增加或不变，如果吃水减少则说明该船只在这个站点卸货了，吃水增加则说明该船只在这个站点装货，吃水若不变则说明该船只仅仅只是停靠在该站点。通过每个站点中所有船只的吃水变化，看是三种变化中的哪种变化最多，则可以分析出该站点的类型。

> 比如若是吃水不变最多，说明该站点不是LNG站点，而是被用作船只停靠。否则则说明其是LNG站点，是LNG站点后进一步比较吃水减少和吃水增多的船只数量，分析出其是进口站点还是出口站点。

```python
# 对183个站点进行分析处理
in_rate = []
out_rate = []
no_rate = []
for i in range(0, 183):
    # 第一个索引是分组编号，第二个索引为0为分组标号，为1是分组的数据
    g = list(newgroup)[i][1]
    # 船最早进来的数据
    firstin = g.groupby("mmsi", as_index=False).agg({'时间': 'min'})
    # 加上其入站的吃水深度
    firstin = pd.merge(firstin, g)
    # 删除不用的项目，只保留mmsi、入站吃水
    firstin = firstin.drop(columns=["时间", "经度", "纬度", "站点编号"])
    # 改名
    firstin.columns = ["mmsi", "入站吃水"]
    # 船最晚出去的数据
    lastout = g.groupby("mmsi", as_index=False).agg({'时间': 'max'})
    # 加上其入站的吃水深度
    lastout = pd.merge(lastout, g)
    # 删除不用的项目，只保留mmsi、入站吃水
    lastout = lastout.drop(columns=["时间", "经度", "纬度", "站点编号"])
    # 改名
    lastout.columns = ["mmsi", "出站吃水"]
    # 将入站吃水和出战吃水合并
    res = pd.merge(firstin, lastout)
    res["变化(m)"] = (res["出站吃水"] - res["入站吃水"]) / 10
    temp = []
    for i in range(0, res.shape[0]):
        if res['变化(m)'][i] < 0:
            temp.append(-1)  # 卸货
        elif res['变化(m)'][i] == 0:
            temp.append(0)  # 停靠
        else:
            temp.append(1)  # 装货
    res["变化行为"] = temp
    # 计算船只变化行为的比率
    boat_rate = res["变化行为"].value_counts(normalize=True)  # boat_rate返回回来为Series类型
    if -1 in boat_rate:
        out_rate.append(boat_rate[-1])
    else:
        out_rate.append(0)
    if 0 in boat_rate:
        no_rate.append(boat_rate[0])
    else:
        no_rate.append(0)
    if 1 in boat_rate:
        in_rate.append(boat_rate[1])
    else:
        in_rate.append(0)
```

我们输出了中间文件，用比率来描述出了每个站点的三种变化船只。

	code	latitude	longitude	in_rate	out_rate	no_rate
2	1	-22.78699148	-43.13067113	0	0.771428571	0.228571429
3	2	25.12463352	119.013107	0.055555556	0.611111111	0.333333333
4	3	35.89028339	14.85538811	0.117647059	0.147058824	0.735294118
5	4	29.69508532	-93.6658821	0.787878788	0.060606061	0.151515152
6	5	25.85162851	51.79970761	0.534883721	0.162790698	0.302325581
7	6	-20.86548499	115.577603	0.310810811	0.216216216	0.472972973
8	7	8.834234928	-79.50800478	0.262626263	0.191919192	0.545454545

最终确定每个站点的类型。（划分规则在分析结果中）

code	latitude	ongitude	is_LNG	IN
1	-22.787	-43.1307	true	false
2	25.12463	119.0131	true	false
3	35.89028	14.85539	false	None
4	29.69509	-93.6659	true	true
5	25.85163	51.79971	true	true
6	-20.8655	115.5776	true	true
7	8.834235	-79.508	true	true
8	-22.8144	14.45374	true	true
9	36.90267	-6.87674	true	false
10	3.348159	112.8699	true	true
11	-38.788	-62.2909	true	false
12	33.83487	130.9626	true	false
13	-20.1491	57.43743	false	None
14	70.96039	25.75806	false	None
15	51.98599	3.889204	true	false
16	-2.418	132.8871	true	true
17	4.289703	73.5148	false	None
18	71.31713	72.34473	true	true
19	3.121508	9.707194	true	true
20	-23.815	151.4103	true	true
21	31.42018	32.23087	true	true
22	-3.47712	-38.7232	false	None
23	36.8246	-75.3559	true	true
24	1.3	100.55	false	None
25	58.82351	17.95909	true	false
26	10.48317	-61.6838	true	true
27	-32.9079	-71.5586	true	false
28	43.18451	141.2812	true	false

分析结果

LNG站点分析聚类结果

查看压缩包中lng_results_list.json文件

lng_results_list.json > {} 172


```
[
    {
        "code": 1,
        "latitude": -22.7869914847,
        "longitude": -43.1306711276,
        "is_LNG": "true",
        "IN": "false"
    },{
        "code": 2,
        "latitude": 25.1246335229,
        "longitude": 119.01310703,
        "is_LNG": "true",
        "IN": "false"
    },{
        "code": 3,
        "latitude": 35.8902833913,
        "longitude": 14.8553881096,
        "is_LNG": "false",
        "IN": "None"
    }, {
        "code": 4,
        "latitude": 29.6950853212,
        "longitude": -93.6658821017,
        "is_LNG": "true",
        "IN": "true"
    }, {
        "code": 5,
        "latitude": 25.8516285093,
        "longitude": 51.7997076143,
        "is_LNG": "true",
        "IN": "true"
    },{
```


划分规则

- 泊点：当站点中的停靠船只数量比率达到70%
- 出站：站点中装货船只数量比率大于入站船只比率
- 入站：站点中卸货船只数量比率大于入站船只比率

统计信息

```
lng = data["is_LNG"].value_counts()
lng
```

[8] ✓ 0.4s

```
True     129
False     54
Name: is_LNG, dtype: int64
```

```
in_out = data["IN"].value_counts()
in_out
```

[9] ✓ 0.8s

```
false    74
true     55
None     54
Name: IN, dtype: int64
```

经过统计

- lng站点：129个
  - 进口站点：55个
  - 出口站点：74个
- 泊点：54个